

REPORT

Aim

To debug the brute force Pair Sum algorithm, identify the premature return statement, correct the logic, optimize redundant comparisons, and analyze its time complexity.

Problem Statement

The given brute force Pair Sum implementation contains a logical error because the `return None` statement is placed inside the outer loop.

As a result, the algorithm terminates after checking only the first iteration of the outer loop, preventing many valid pairs from being examined.

The program must be corrected so that all possible pairs are checked before concluding that no solution exists.

Theory

The Pair Sum problem requires finding two elements whose sum equals a given target.

The brute force approach compares every possible pair.

The incorrect Python code is

```
def pair_sum(arr, target):  
    for i in range(len(arr)):  
        for j in range(i+1, len(arr)):  
            if arr[i] + arr[j] == target:  
                return (i, j)  
    return None
```

The statement

```
return None
```

is inside the outer loop.

Therefore, after checking only the first value of `i`, the function immediately exits if no pair is found.

Error Analysis

Suppose

```
Array = [2, 7, 11, 15, 3, 6]
Target = 9
```

The algorithm checks

```
2 + 7
2 + 11
2 + 15
2 + 3
2 + 6
```

If none matched, it would execute

```
return None
```

without checking

```
7 + 11
7 + 15
7 + 3
...
```

Hence many valid pairs are never considered.

Corrected Algorithm

1. Read the array.
2. Read the target.
3. Select one element using the outer loop.
4. Compare it with every remaining element.
5. If a pair is found, print the indices.
6. Continue checking until every combination is examined.
7. If no pair exists, print "No pair found."

Optimization

Instead of

(i, j)

and

(j, i)

being compared separately,

the inner loop begins from

$j = i + 1$

This avoids duplicate comparisons while preserving the brute-force method.

Example:

Instead of checking

$2+7$

$7+2$

only

$2+7$

is checked.

Corrected C Program

```
#include <stdio.h>
```

```
// Function to find the pair whose sum equals the target
```

```
void pairSum(int arr[], int size, int target)
```

```
{
```

```
    int i, j;
```

```
    int found = 0;
```

```
    // Outer loop selects the first element
```

```

for(i = 0; i < size; i++)
{
    // Inner loop starts from i+1 to avoid
    // duplicate and mirror comparisons
    for(j = i + 1; j < size; j++)
    {
        if(arr[i] + arr[j] == target)
        {
            printf("Pair found at indices (%d, %d)\n", i, j);
            printf("Values are %d and %d\n", arr[i], arr[j]);
            found = 1;
            return;    // Stop after finding the first valid pair
        }
    }
}

// Executed only if no pair is found
if(found == 0)
{
    printf("No pair found.\n");
}

int main()
{
    int arr[] = {2, 7, 11, 15, 3, 6};
    int size = sizeof(arr) / sizeof(arr[0]);
    int target;

    printf("Enter target sum: ");
    scanf("%d", &target);

    pairSum(arr, size, target);

    return 0;
}

```

Output

Case 1

Enter target sum: 9

Pair found at indices (0,1)
Values are 2 and 7

Case 2

Enter target sum: 20

No pair found.

Time Complexity

Outer loop

$O(n)$

Inner loop

$O(n)$

Overall

$O(n^2)$

Space Complexity

Only a few variables are used.

$O(1)$

Advantages

- Simple to understand.
 - Easy to implement.
 - Correctly checks all possible pairs.
 - Eliminates redundant mirror comparisons.
-

Disadvantages

- Slow for large datasets.

- Time complexity is quadratic.
-

Applications

- Two Sum Problem
 - Database Record Matching
 - Financial Transaction Analysis
 - Cybersecurity Event Correlation
 - Data Mining
 - Search Algorithms
-

Result

The logical error caused by the premature `return None` statement was successfully identified and corrected. The revised algorithm checks all valid element pairs before concluding failure and avoids redundant mirror comparisons by starting the inner loop from `i + 1`. The corrected brute-force algorithm has a **time complexity of $O(n^2)$** and a **space complexity of $O(1)$** .